# Razorpay Payment Integration - Complete Documentation

## 📁 Key Files Overview

### Backend Files

- **PaymentController.java** - Main payment controller handling order creation and verification
- **Booking.java** - Entity with Razorpay payment fields
- **EmailService.java** - Sends payment confirmation emails
- **BookingRepository.java** - Database queries for payment records

### Frontend Files

- **Payment.jsx** - Payment UI component with Razorpay integration

### Configuration

- **pom.xml** - Razorpay SDK dependency (v1.4.6)
- **application.yml** - Email configuration for confirmations

---

## 🎯 Architecture Overview

### Payment Flow Model

```
Server-Side Order Creation → Client-Side Payment UI → Server-Side Signature
Verification → Email Confirmation
```

### Technology Stack

- **Backend**: Spring Boot 3.2, Java 17, Razorpay Java SDK 1.4.6
- **Frontend**: React 19, Razorpay Checkout.js (CDN)
- **Payment Gateway**: Razorpay
- **Email Service**: Spring Mail (Gmail SMTP)
- **Database**: MySQL 8 (AWS RDS)

---

## 📊 Complete Payment Flow - Step by Step

### STEP 1: User Initiates Payment (Frontend)

**Route**: `/payment/:bookingId`

**Component**: `Payment.jsx`

**Process**:

```
// Component mounts and auto-triggers payment
useEffect(() => {
    loadBookingAndInitiatePayment();
}, [bookingId]);

const loadBookingAndInitiatePayment = async () => {
    // 1. Call backend to create Razorpay order
    const res = await API.post(`/api/payment/create-order/${bookingId}`);
    const orderData = res.data;

    // 2. Load Razorpay checkout.js script dynamically
    const script = document.createElement('script');
    script.src = 'https://checkout.razorpay.com/v1/checkout.js';
    script.async = true;
    script.onload = () => openRazorpay(orderData);
    document.body.appendChild(script);
};
```

**Why Dynamic Script Loading?**

- Reduces initial bundle size
- Always fetches latest version from Razorpay CDN
- Avoids bundling external dependencies

---

## STEP 2: Create Razorpay Order (Backend)

**Endpoint**: `POST /api/payment/create-order/{bookingId}`

**Controller**: `PaymentController.java`

**Code Flow**:

```
@PostMapping("/create-order/{bookingId}")
public ResponseEntity<?> createOrder(@PathVariable Long bookingId) {
    // 1. Fetch booking from database
    Booking booking = bookingRepository.findById(bookingId)
            .orElseThrow(() -> new RuntimeException("Booking not found"));

    // 2. Initialize Razorpay client with credentials
    RazorpayClient razorpayClient = new RazorpayClient(razorpayKeyId,
razorpayKeySecret);

    // 3. Create order request (amount in paise)
    JSONObject orderRequest = new JSONObject();
    orderRequest.put("amount", (int)(booking.getAmount() * 100)); // ₹500 = 50000
paise
    orderRequest.put("currency", "INR");
    orderRequest.put("receipt", "booking_" + bookingId);
```

```
    // 4. Call Razorpay API to create order
    Order order = razorpayClient.orders.create(orderRequest);

    // 5. Save order ID to booking (crucial for later verification)
    booking.setRazorpayOrderId(order.get("id"));
    booking.setPaymentStatus("PENDING");
    bookingRepository.save(booking);

    // 6. Log payment initiation
    actionLogger.logClientAction(
        bookingId.intValue(),
        email,
        "PAYMENT_INITIATED",
        String.format("OrderID: %s, Amount: %.2f", order.get("id"),
booking.getAmount())
    );

    // 7. Return order details + user info to frontend
    return ResponseEntity.ok(Map.of(
        "orderId", order.get("id"),
        "amount", booking.getAmount(),
        "currency", "INR",
        "keyId", razorpayKeyId,
        "bookingId", bookingId,
        "tenantName", booking.getTenantName(),
        "tenantEmail", booking.getTenantEmail(),
        "tenantPhone", booking.getTenantPhone()
    ));
}
```

**Critical Points**:

- ☑ Amount converted to **paise** (multiply by 100)
- ☑ `razorpayOrderId` saved to database immediately
- ☑ Payment status set to **PENDING**
- ☑ Only `keyId` (public key) sent to frontend, `keySecret` remains on server

---

### STEP 3: Open Razorpay Checkout Modal (Frontend)

**Function**: `openRazorpay(orderData)`

**Code**:

```
const openRazorpay = (orderData) => {
    const options = {
        key: orderData.keyId,              // Razorpay public key
        amount: orderData.amount * 100,    // Amount in paise
        currency: "INR",
        name: "PG Finder",
        description: `Booking Payment - #${bookingId}`,
```

```
        order_id: orderData.orderId,       // ☑ CRITICAL: order_id from backend

        // Pre-fill user details for better UX
        prefill: {
            name: orderData.tenantName,
            email: orderData.tenantEmail,
            contact: orderData.tenantPhone
        },

        theme: {
            color: "#6366f1"  // Custom brand color
        },

        // Success handler - triggered after successful payment
        handler: async function (response) {
            await verifyPayment(response);
        },

        // Cancel/Dismiss handler
        modal: {
            ondismiss: function() {
                toast.info("Payment cancelled");
                navigate("/client");
            }
        }
    };

    // Initialize and open Razorpay checkout
    const razorpay = new window.Razorpay(options);
    razorpay.open();
};
```

**What Happens in Razorpay Modal?**

1. User selects payment method (UPI, Cards, Net Banking, Wallets)
2. Completes payment on Razorpay's secure interface
3. On success, Razorpay returns these 3 values:
    - `razorpay_order_id` - Order ID we created
    - `razorpay_payment_id` - Unique payment transaction ID
    - `razorpay_signature` - HMAC SHA256 hash for verification

---

## STEP 4: Verify Payment Signature (Backend) 🔒

**Endpoint**: `POST /api/payment/verify-payment`

**Controller**: `PaymentController.java`

**This is the MOST CRITICAL security step!**

**Code Flow**:

```
@PostMapping("/verify-payment")
public ResponseEntity<?> verifyPayment(@RequestBody Map<String, String>
paymentData) {
    String razorpayOrderId = paymentData.get("razorpay_order_id");
    String razorpayPaymentId = paymentData.get("razorpay_payment_id");
    String razorpaySignature = paymentData.get("razorpay_signature");

    // 1. ☑ VERIFY SIGNATURE using Razorpay utility
    JSONObject options = new JSONObject();
    options.put("razorpay_order_id", razorpayOrderId);
    options.put("razorpay_payment_id", razorpayPaymentId);
    options.put("razorpay_signature", razorpaySignature);

    boolean isValidSignature = Utils.verifyPaymentSignature(options,
razorpayKeySecret);

    if (isValidSignature) {
        // 2. Find booking by razorpayOrderId
        Booking booking = bookingRepository.findByRazorpayOrderId(razorpayOrderId)
                .orElseThrow(() -> new RuntimeException("Booking not found"));

        // 3. Update booking with payment details
        booking.setRazorpayPaymentId(razorpayPaymentId);
        booking.setRazorpaySignature(razorpaySignature);
        booking.setPaymentStatus("PAID");
        booking.setStatus("CONFIRMED");
        bookingRepository.save(booking);

        // 4. Log successful payment
        actionLogger.logClientAction(
            booking.getId().intValue(),
            email,
            "PAYMENT_SUCCESS",
            String.format("PaymentID: %s, Amount: %.2f", razorpayPaymentId,
booking.getAmount())
        );

        // 5. Send confirmation email
        try {
            emailService.sendPaymentConfirmationEmail(booking, razorpayPaymentId);
            log.info("Payment confirmation email sent to: {}",
booking.getTenantEmail());
        } catch (Exception emailException) {
            log.error("Failed to send email: {}", emailException.getMessage());
            // Continue - email failure shouldn't fail payment confirmation
        }

        return ResponseEntity.ok(Map.of(
            "success", true,
            "message", "Payment verified successfully",
            "bookingId", booking.getId()
        ));
    } else {
```

```
        log.error("Invalid payment signature for order: {}", razorpayOrderId);
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(Map.of("success", false, "message", "Invalid payment
signature"));
    }
}
```

---

### How Signature Verification Works 🔐

**Algorithm**: HMAC SHA256

**Formula**:

```
Expected Signature = HMAC_SHA256(
    razorpay_order_id + "|" + razorpay_payment_id,
    razorpay_key_secret
)

If (Expected Signature == Received razorpay_signature)
    → Payment is GENUINE ☑
Else
    → Payment is FAKE/TAMPERED ❌
```

**Why This is Critical**:

- Without signature verification, anyone could call `/verify-payment` with fake payment IDs
- The secret key **NEVER leaves the server**, making it cryptographically impossible to forge
- Even if someone intercepts the response, they can't recreate the signature without the secret

---

### STEP 5: Send Payment Confirmation Email

**Service**: `EmailService.java`

**Method**: `sendPaymentConfirmationEmail(Booking booking, String razorpayPaymentId)`

**Email Contents**:

```
✓ Payment Successful!

Dear [Client Name],

Your payment has been successfully processed.

Booking Details:
- Booking ID: #12345
- Client Name: John Doe
- PG Name: Sunrise Residency
```

```
- Amount Paid: ₹5,000.00
- Check-in Date: 15 Mar 2026, 02:00 PM
- Payment ID: pay_NXjsJzG7S8KLCr

Thank you for choosing Residentia!
```

**Technology**:

- Spring Mail with MimeMessageHelper
- HTML email with CSS styling
- Gmail SMTP (port 587, TLS enabled)

---

## 💾 Database Schema - Payment Fields

**Table**: `bookings`

**Payment-Related Columns**:

```java
@Column(name = "razorpay_order_id")
private String razorpayOrderId;         // Created during order creation

@Column(name = "razorpay_payment_id")
private String razorpayPaymentId;       // Received after payment success

@Column(name = "razorpay_signature")
private String razorpaySignature;       // Stored for audit trail

@Column(name = "payment_status")
private String paymentStatus;           // PENDING / PAID / FAILED
```

**Custom Repository Method**:

```java
Optional<Booking> findByRazorpayOrderId(String razorpayOrderId);
```

---

## 🔒 Security Measures Implemented

| Security Feature | Implementation | Purpose |
|---|---|---|
| **Server-side order creation** | Orders created on backend only | Prevents amount tampering |
| **Signature verification** | HMAC SHA256 using secret key | Prevents fake payment confirmations |
| **Secret key protection** | Stored in env variables, never exposed | Keeps cryptographic key secure |

| Security Feature | Implementation | Purpose |
|---|---|---|
| **HTTPS enforcement** | All API calls over HTTPS | Prevents man-in-the-middle attacks |
| **JWT authentication** | Payment endpoints require valid JWT | Only authenticated users can pay |
| **Audit logging** | All payment events logged | Forensic analysis and monitoring |
| **Database-driven verification** | Order ID stored before payment | Links payment to valid booking |

## 📦 Dependencies

### Backend (pom.xml)

```
<!-- Razorpay SDK -->
<dependency>
    <groupId>com.razorpay</groupId>
    <artifactId>razorpay-java</artifactId>
    <version>1.4.6</version>
</dependency>

<!-- JSON Library for Razorpay -->
<dependency>
    <groupId>org.json</groupId>
    <artifactId>json</artifactId>
    <version>20231013</version>
</dependency>

<!-- Spring Mail for email confirmations -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

### Frontend

```
{
  "dependencies": {
    "axios": "^1.13.0",
    "react-toastify": "^latest"
  }
}
```

**Razorpay Checkout**: Loaded dynamically via CDN (`https://checkout.razorpay.com/v1/checkout.js`)

---

## 🎨 Payment Flow Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                          RAZORPAY PAYMENT FLOW                       │
└──────────────────────────────────────────────────────────────────────┘

┌──────────┐  1. Navigate to        ┌──────────────┐
│  Client  │  /payment/:bookingId   │  Payment.jsx │
│  Browser │ ──────────────────────>│  (React)     │
└──────────┘                        └──────────────┘
                                          │
                                          │
                              2. POST /api/payment/create-order/{bookingId}
                                          │
                                          ▼
                          ┌──────────────────────────────┐
                          │  PaymentController.java      │
                          │  ──────────────────────      │
                          │  • Fetch Booking from DB     │
                          │  • Create Razorpay Order     │
                          │  • Save razorpayOrderId      │
                          │  • Set paymentStatus = PENDING │
                          │  • Return order details      │
                          └──────────────────────────────┘
                                          │
                                          │
                                3. Open Razorpay Modal
                                          │
┌─────────────────────────────────────────┼────────────────────────────┐
│                                         │                            │
│                                         │                            │
│              ┌──────────────────────────▼──────────────────────┐    │
│              │  Razorpay Checkout Modal (Third-party)          │    │
│              │  ──────────────────────────────────────         │    │
│              │  • UPI                                          │    │
│              │  • Credit/Debit Cards                           │    │
│              │  • Net Banking                                  │    │
│              │  • Wallets (Paytm, PhonePe, etc.)               │    │
│              └─────────────────────────────────────────────────┘    │
│                                         │                            │
│                                         │                            │
│                  4. User Completes Payment                           │
│                                         │                            │
│              ┌──────────────────────────▼──────────────────────┐    │
│              │  Razorpay Returns to handler():                 │    │
│              │  • razorpay_order_id                            │    │
│              │  • razorpay_payment_id                          │    │
│              │  • razorpay_signature (HMAC-SHA256)             │    │
│              └─────────────────────────────────────────────────┘    │
│                                         │                            │
└─────────────────────────────────────────┼────────────────────────────┘
                                          │
                                          │
                          5. POST /api/payment/verify-payment
                             (order_id, payment_id, signature)
                                          │
                                          ▼
                          ┌──────────────────────────────┐
```

```
                              │  PaymentController.java          │
                              │  ────────────────────────        │
                              │  • Verify HMAC-SHA256 Signature  │  ⚡ CRITICAL
                              │  • Find Booking by orderId       │
                              │  • Update razorpayPaymentId      │
                              │  • Set paymentStatus = PAID      │
                              │  • Set bookingStatus = CONFIRMED │
                              │  • Call EmailService             │
                              └──────────────────────────────────┘
                                              │
                                              │
                                 6. Send Confirmation Email
                                              │
                                              ▼
                              ┌──────────────────────────────────┐
                              │  EmailService.java               │
                              │  ────────────────────────        │
                              │  • Create HTML email             │
                              │  • Attach payment details        │
                              │  • Send via Gmail SMTP           │
                              └──────────────────────────────────┘
```

---

## 🎤 Interview Discussion Points

### Q1: "Walk me through your payment integration."

**Answer**:

> "I integrated Razorpay using a secure 3-step flow: Order Creation, Payment Processing, and Signature Verification.
>
> First, when a user clicks 'Pay Now', the frontend calls our backend to create a Razorpay order. The backend generates an order via Razorpay's API, stores the order ID in our database, and returns it to the frontend along with the public key.
>
> Second, the frontend loads Razorpay's checkout modal dynamically, which handles all payment methods. Once the user completes payment, Razorpay returns three critical values: order ID, payment ID, and a signature.
>
> Third, and most importantly, the frontend sends these to our backend for signature verification. We use HMAC SHA256 to verify the signature against our secret key. Only after successful verification do we mark the booking as PAID and send a confirmation email."

---

### Q2: "How did you ensure payment security?"

**Answer**:

> "I implemented multiple layers of security:
>
> 1. **Server-side order creation**: Orders are created only on the backend, so users can't tamper with the amount.

> 2. **Signature verification**: This is crucial. Razorpay sends a HMAC SHA256 signature that we verify using our secret key. The secret never leaves the server, making it cryptographically impossible to fake a payment.
> 3. **JWT authentication**: All payment endpoints require valid JWT tokens, so only authenticated users can initiate payments.
> 4. **Database-driven validation**: We store the Razorpay order ID immediately after creation, and during verification, we query by this ID to ensure the payment corresponds to a real booking.
> 5. **Audit logging**: Every payment attempt, success, and failure is logged with user email, timestamp, and amount for forensic analysis."

---

### Q3: "Why did you load Razorpay script dynamically?"

**Answer**:

> "Three main reasons:
>
> 1. **Reduced bundle size**: By not including it in our main bundle, we keep the initial page load faster.
> 2. **Always latest version**: Loading from Razorpay's CDN ensures we automatically get security patches and updates.
> 3. **On-demand loading**: The script only loads when users reach the payment page, not on every route."

---

### Q4: "How do you handle payment failures?"

**Answer**:

> "We track payment status in the database with three states: PENDING, PAID, and FAILED.
>
> When a user initiates payment, the status is set to PENDING. If signature verification fails, we don't update the status, so it remains PENDING. The booking stays in PENDING state, and the user can retry payment from their dashboard.
>
> We also distinguish between different failure scenarios:
>
> - **User cancellation**: Caught by the `ondismiss` handler in the modal
> - **Signature verification failure**: Logged as PAYMENT_FAILED with detailed error messages
> - **Network errors**: Caught by try-catch blocks with user-friendly toast notifications
>
> All failures are logged via our ActionLogger service for admin review and analytics."

---

### Q5: "What about refunds?"

**Answer**:

> "Currently, refunds aren't implemented in the production code, but the architecture supports it easily. Here's how I would add it:
>
> 1. Create a new endpoint: `POST /api/payment/refund/{paymentId}`
> 2. Add role-based access control to ensure only admins can initiate refunds
> 3. Use Razorpay SDK's refund API:
>
> ```
> Payment payment = razorpayClient.payments.fetch(paymentId);
> JSONObject refundRequest = new JSONObject();
> refundRequest.put("amount", amountInPaise);
> Refund refund = payment.refund(refundRequest);
> ```
>
> 4. Update booking status to REFUNDED and store refund ID
> 5. Send refund confirmation email to the customer
>
> The Razorpay SDK makes this straightforward since all payment objects have a built-in `refund()` method."

---

### Q6: "How do you test payment integration?"

**Answer**:

> "I use Razorpay's test mode with test API keys. Razorpay provides test card numbers and UPI IDs that simulate different scenarios:
>
> - **Successful payment**: Card `4111 1111 1111 1111`
> - **Failed payment**: Card `4000 0000 0000 0002`
> - **3D Secure authentication**: Card `5267 3181 8797 5449`
>
> For automated testing, I would:
>
> 1. Mock the RazorpayClient class in unit tests
> 2. Test signature verification logic with known inputs
> 3. Test order creation with valid/invalid booking IDs
> 4. Verify database state changes after successful/failed payments
>
> In production, we monitor payment success rates, average transaction times, and failure patterns through our logging system."

---

### Q7: "What happens if the database save fails after payment?"

**Answer**:

> "This is a critical scenario. Currently, if the database save fails after signature verification, the payment is successful on Razorpay's side, but our system doesn't reflect it.

> To handle this, I would implement:
>
> 1. **Transaction management**: Wrap the verification logic in a `@Transactional` block so database operations roll back on failure.
> 2. **Webhook fallback**: Implement Razorpay's webhook endpoint (`/payment/webhook`) that receives payment confirmations asynchronously. If our verification endpoint fails, the webhook can retry updating the booking.
> 3. **Reconciliation job**: A scheduled job that queries Razorpay API for successful payments in the last 24 hours and cross-references with our database to catch any missed updates.
> 4. **Idempotency**: Ensure the verification endpoint is idempotent, so multiple calls with the same payment ID don't cause issues.
>
> This creates a fail-safe system where even if one mechanism fails, others catch it."

---

## ⚠ Current Limitations & Future Enhancements

### Limitations

1. **No webhook implementation**: Currently relies solely on frontend verification flow
2. **No refund functionality**: Refunds must be processed manually via Razorpay dashboard
3. **Single currency support**: Only INR supported
4. **No payment retry mechanism**: Users must manually retry failed payments

### Future Enhancements

1. **Razorpay Webhooks**:
   - Implement `/payment/webhook` endpoint for asynchronous confirmations
   - Adds redundancy if frontend verification fails
2. **Payment Analytics Dashboard**:
   - Success/failure rates
   - Average transaction value
   - Payment method breakdown
3. **Automatic Refund System**:
   - Admin panel for refund approvals
   - Partial refund support
   - Refund status tracking
4. **Payment Reminders**:
   - Email reminders for PENDING payments
   - SMS notifications via Twilio
5. **Multiple Payment Gateways**:

  - Stripe as fallback
  - Let users choose preferred gateway

---

## 📊 Razorpay API Credentials Configuration

**Configuration Method**: Spring Boot `@Value` annotation

```
@Value("${razorpay.key.id:rzp_test_xxxxxxxxxx}")
private String razorpayKeyId;

@Value("${razorpay.key.secret:xxxxxxxxxx}")
private String razorpayKeySecret;
```

**For Production**:

- Store credentials in environment variables
- Never commit to Git
- Use AWS Secrets Manager or HashiCorp Vault
- Rotate keys periodically

---

## 🔧 Troubleshooting Common Issues

| Issue | Cause | Solution |
|---|---|---|
| "Invalid signature" error | Mismatched secret key or corrupted data | Verify secret key matches Razorpay dashboard; check data isn't modified in transit |
| Payment modal doesn't open | Razorpay script failed to load | Check browser console; verify CDN URL; test network connectivity |
| Email not sent | SMTP authentication failure | Verify Gmail app password; check firewall rules for port 587 |
| Amount mismatch | Frontend-backend currency conversion | Always multiply by 100 on backend, Razorpay expects paise |
| "Order not found" | Order ID not saved before payment | Ensure database save succeeds before returning order to frontend |

---

## 📚 Additional Resources

- **Razorpay API Documentation**: https://razorpay.com/docs/api/
- **Razorpay Java SDK**: https://github.com/razorpay/razorpay-java
- **Razorpay Checkout.js**: https://razorpay.com/docs/payments/payment-gateway/web-integration/standard/
- **HMAC SHA256 Signature Verification**: https://razorpay.com/docs/payments/payment-gateway/web-integration/standard/verify-payment-signature/

---

## 👨‍💻 Developer Notes

**Last Updated**: March 1, 2026
**Razorpay SDK Version**: 1.4.6
**Checkout.js Version**: Latest (CDN)
**Spring Boot Version**: 3.2.0
**Java Version**: 17

**Author**: Prajwal (PG Finder Project)

---

## ☑ Pre-Production Checklist

- [x] Signature verification implemented
- [x] Payment status tracking in database
- [x] Email confirmations sent
- [x] Error handling for all endpoints
- [x] Audit logging implemented
- [x] JWT authentication on payment endpoints
- [ ] Webhook endpoint implemented
- [ ] Refund functionality added
- [ ] Payment reconciliation job scheduled
- [ ] Load testing completed
- [ ] Security audit performed
- [ ] Production API keys configured

---

**End of Documentation**